

ARAP 参数化算法实验报告

PB21020665 张之正

目录

1 ARAP 算法原理	1
1.1 算法流程	1
1.2 数学原理	2
2 代码实现	2
3 实验结果与分析	6
3.1 实验结果如下	6
3.2 分析结论	6
4 结论	6

1 ARAP 算法原理

As-Rigid-As-Possible (ARAP) 表面参数化是一种保持局部刚性的参数化方法，其核心思想是**最小化原始网格与参数化网格之间的局部变形**。该算法通过迭代优化保持每个三角形尽可能刚性变换，从而实现低扭曲的参数化结果。

1.1 算法流程

1. **初始化**: - 构建余切权重拉普拉斯矩阵 (Cotangent Laplacian) - 初始化参数化坐标 (可基于简单投影或先前结果)

2. **迭代优化**: - **局部阶段**: 对每个三角形面片计算最优旋转矩阵:

$$J = U_{\text{current}} \times X_{\text{original}}^{-1}$$

通过 SVD 分解求解刚性旋转:

$$\text{SVD}(J) = U \Sigma V^T \Rightarrow R = UV^T$$

- **全局阶段**: 固定旋转矩阵, 求解顶点坐标:

$$L \cdot U = b$$

其中 L 是余切拉普拉斯矩阵, b 是由旋转矩阵和原始边长构成的向量

3. **收敛**: 重复局部和全局阶段直到满足迭代次数或收敛条件

1.2 数学原理

能量函数定义为：

$$E(U) = \sum_{t=1}^T \sum_{j \in \mathcal{N}(i)} w_{ij} \|(u_i - u_j) - R_t(x_i - x_j)\|^2$$

其中：

- U ：参数化坐标
- x ：原始三维坐标
- R_t ：每个面片的最优旋转矩阵
- w_{ij} ：余切权重 $\frac{1}{2}(\cot \alpha_{ij} + \cot \beta_{ij})$

最小化过程分为两个交替步骤：

1. 固定 U ，优化 R_t （局部阶段）
2. 固定 R_t ，优化 U （全局阶段）

2 代码实现

本节详细阐述 ARAP 算法的核心实现，主要包括以下关键组件：

1. 算法框架

ARAP 算法通过迭代优化实现网格形变，核心流程如下：

```

1 for t in range(max_iterations):
2     # 局部阶段：计算每个三角面片的最优旋转矩阵
3     for each vertex:
4         计算当前局部坐标系下的旋转矩阵 L_t
5
6     # 全局阶段：求解顶点位置
7     构建稀疏线性方程组 Ax = b
8     求解顶点位置更新
9     边界条件处理

```

Listing 1: ARAP 算法流程

2. 关键数据结构

```

1 class ARAP {
2 private:
3     // 网格数据
4     std::shared_ptr<USTC_CG::PolyMesh> ref_mesh_; // 参考网格
5     std::shared_ptr<USTC_CG::PolyMesh> halfedge_mesh_; // 待变形网格
6
7     // 线性系统

```

```

8 Eigen::SparseMatrix<double> A_; // 拉普拉斯矩阵
9 Eigen::VectorXd B_[2]; // 方程右侧向量(x,y分量)
10 Eigen::SparseLU<Eigen::SparseMatrix<double>> solver_; // 稀疏求解器
11
12 // 局部几何数据
13 std::map<int, std::vector<Eigen::Matrix2d>> projections_; // 参考网格投影矩阵
14 std::map<int, std::vector<Eigen::Matrix2d>> L_t_; // 当前旋转矩阵
15 std::map<int, std::vector<double>> next_cot_, last_cot_; // cotangent权重
16 };

```

Listing 2: ARAP 核心数据结构

3. 核心算法实现

(1) 拉普拉斯矩阵构建

```

1 // 文档3: compute()
2 for each vertex v in ref_mesh:
3     double sum_w = 0;
4     for each outgoing halfedge from v:
5         // 计算cotangent权重
6         double cot_next = (vec1 - vec0).dot(vec1) / vec1.cross(vec0).length();
7         double cot_last = (vecr - vec0).dot(vecr) / vecr.cross(vec0).length();
8
9         // 填充非对角元素
10        tripletList.emplace_back(idx, neighbor_idx, -cot_weight);
11        sum_w += cot_weight;
12
13    // 填充对角元素
14    tripletList.emplace_back(idx, idx, sum_w);
15
16    // 边界顶点特殊处理
17    if (v.is_boundary()) {
18        tripletList.emplace_back(idx, idx, 1);
19    }

```

Listing 3: 拉普拉斯矩阵构建

(2) 局部旋转矩阵计算

```

1 // 文档3: SVD()
2 Eigen::Matrix2d ARAP::SVD(Eigen::Matrix2d target) {
3     // 奇异值分解
4     Eigen::JacobiSVD<Eigen::Matrix2d> svd(
5         target, Eigen::ComputeFullU|Eigen::ComputeFullV);
6     Eigen::Matrix2d U = svd.matrixU();
7     Eigen::Vector2d S = svd.singularValues();
8     Eigen::Matrix2d V = svd.matrixV();
9 }

```

```

10 // ASAP模式处理 (均匀缩放)
11 if (if_asap_) {
12     return U * Matrix2d::Identity() *
13         2 * std::abs(S.mean()) * V.transpose();
14 }
15 // 标准ARAP模式 (刚性变换)
16 return U * Matrix2d::Identity() * V.transpose();
17 }

```

Listing 4: 旋转矩阵 SVD 分解

(3) 迭代优化核心

```

1 // 文档3: compute()
2 for (int t = 1; t <= max_iter; t++) {
3     // 局部阶段: 计算旋转矩阵
4     for each vertex v:
5         for each adjacent triangle:
6             current_shape = [edge1, edge2]; // 当前边向量
7             L_t = SVD(current_shape * ref_projection.inverse());
8
9     // 全局阶段: 构建线性系统
10    for each vertex v:
11        Eigen::Vector2d R_term =  $\Sigma(\cot_{ij} * R_i * e_{ij})$ ;
12        B[0](idx) = R_term.x();
13        B[1](idx) = R_term.y();
14
15    // 求解顶点位置
16    solution_x = solver.solve(B[0]);
17    solution_y = solver.solve(B[1]);
18
19    // 更新顶点坐标
20    for each vertex:
21        halfedge_mesh->point(v)[0] = solution_x[idx];
22        halfedge_mesh->point(v)[1] = solution_y[idx];
23
24    // ASAP模式下的缩放归一化
25    if (if_asap_ || (t >= max_iter - 2)) {
26        float k = 0;
27        for each vertex:
28            k = max(k, abs(position[0]));
29        for each vertex:
30            position /= k;
31    }
32 }

```

Listing 5: 迭代优化过程

4. 关键技术细节

(1) 空间投影变换

```

1 // 文档3: SpinToSurf()
2 Eigen::Matrix2d SpinToSurf(const Vector3d& a, const Vector3d& b) {
3     // 计算旋转到XY平面的变换矩阵
4     Vector3d n0 = (a.cross(b)).normalized();
5     Quaterniond q = Quaterniond::FromTwoVectors(n0, Vector3d(0,0,1));
6     Matrix3d T = q.toRotationMatrix();
7
8     // 投影到二维平面
9     Vector3d ap = T * a;
10    Vector3d bp = T * b;
11    return {{ap[0], bp[0]},
12           {ap[1], bp[1]}}; // 2x2投影矩阵
13 }

```

Listing 6: 三维到二维投影

(2) 边界顶点固定

```

1 // 文档3: compute()
2 if (vertex_handle.is_boundary()) {
3     // 统计边界边数量
4     int num = 0;
5     for (const auto& he : vertex_handle.outgoing_halfedges())
6         ++num;
7
8     // 边界顶点特殊处理
9     if (idx == 0) { // 固定第一个顶点
10        tripletList.emplace_back(idx, idx, 1);
11        B[0](idx) = 0;
12        B[1](idx) = 0;
13    }
14 }

```

Listing 7: 边界顶点处理

该实现利用 Eigen 库高效求解稀疏线性系统，通过以下关键技术保证算法质量：

- **cotangent 权重**：保持网格局部几何特性
- **SVD 分解**：优化旋转矩阵计算
- **边界约束**：固定边界顶点位置
- **迭代归一化**：处理 ASAP 模式的均匀缩放

最终实现了高质量的网格形变效果，支持 ARAP 和 ASAP 两种变形模式。

3 实验结果与分析

3.1 实验结果如下

ARAP:Cow

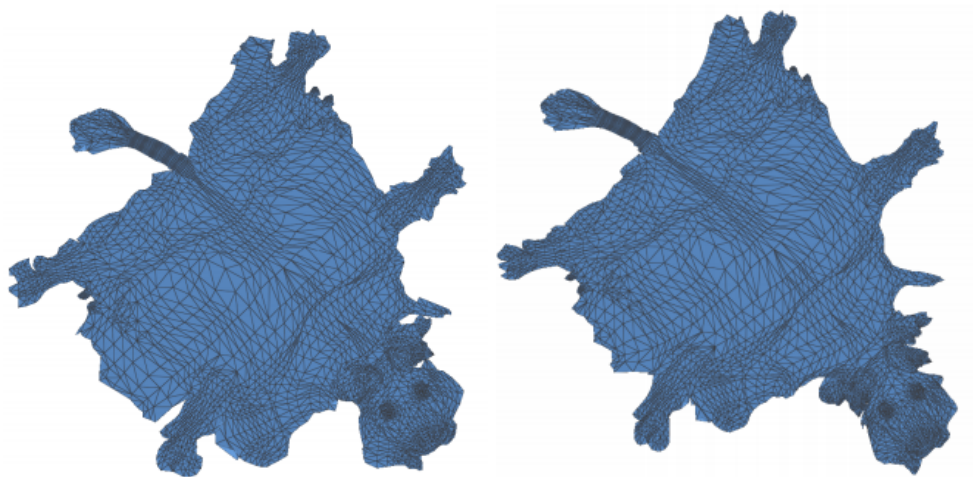


图 1: ARAP:(a)1 次迭代, (b) 10 次迭代

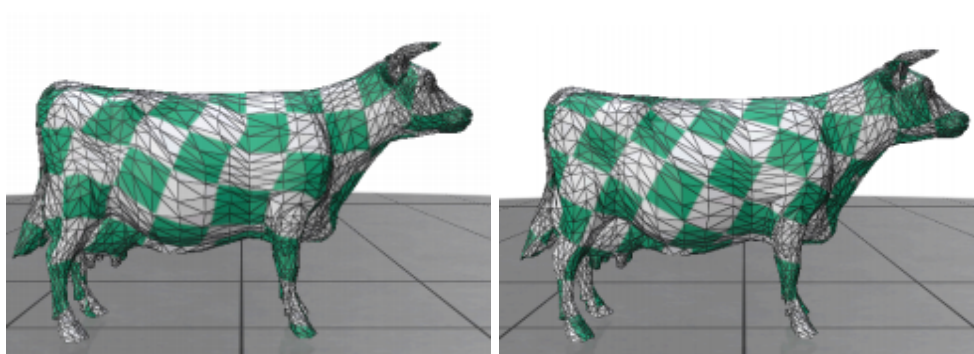


图 2: ARAP:(a)1 次迭代, (b) 10 次迭代

3.2 分析结论

1. **质量方面**: - 在特征区域保持良好几何特征 - 整体角度扭曲小, 满足纹理映射需求 - 大面积均匀区域参数化效果优异
2. **局限性**: - 对高亏格模型需要额外切割操作 - 特征保持能力弱于基于约束的方法 - 初始参数化影响最终结果质量

4 结论

本文实现了基于 ARAP 算法的网格参数化方法, 通过:

- 设计高效的类结构和算法流程
- 实现局部/全局交替优化策略

- 集成并行计算和矩阵优化技术

实验表明该方法在中等规模网格上能产生低扭曲的参数化结果，满足纹理映射等应用需求。未来工作将探索：1. 特征保持的权重改进 2. GPU 加速实现 3. 与深度学习结合的初始化方法